GATE CS — Computer Organization & Architecture

Concise study notes for GATE 2027 · built from the 2016–2026 papers in this repo

Where the marks come from (average marks per paper, 2016–2026)

Topic	Marks / paper	Priority
Cache & memory hierarchy	3.3	Tier 1
Pipelining, hazards, CPI/speed-up	2.0	Tier 2
Instruction set, addressing modes, data path, control	1.5	Tier 2
I/O: DMA, interrupts	0.6	Tier 3

Computer Organization & Architecture carries about **7.5 of the 85 technical marks** in a typical paper. Topics below are listed in the order given in the notes; use the table above to decide where to spend time first.

Legend: **Trap** = common mistake · **Tip** = shortcut. For practice, use `GATE_CS_Computer_Organization.pdf` (past questions + new questions with solutions). These notes were written from general knowledge, not checked line by line against a textbook; verify the less common facts.

1. Cache and Memory Hierarchy

- Hierarchy (fast/small → slow/large): registers, L1, L2, L3, main memory (DRAM), disk. **Locality:** temporal (reuse soon) and spatial (nearby addresses).
- **Address breakdown** (physical address of A bits, block size B bytes, C bytes of cache, associativity k):
  offset bits = $\log_2 B$; number of lines = C/B; number of sets = C/(B·k); index bits = $\log_2$(sets); tag bits = A − index − offset.
  Direct mapped: k = 1 (sets = lines). Fully associative: one set, no index bits (tag = A − offset).
  Example: 32 KB direct-mapped, 64-byte blocks, 32-bit address → offset 6, lines 512, index 9, tag 17.
- Total cache size in bits = lines × (data bits + tag bits + valid bit [+ dirty bit]). Tag-memory overhead counts in such questions.
- **Mapping:** direct (block i → line i mod lines; simple, conflict misses), fully associative (any line; needs parallel tag comparison), k-way set associative (block i → set i mod sets; compromise). Replacement: LRU, FIFO, random.
- **Misses:** compulsory (first reference), capacity (cache too small), conflict (set clash). Larger blocks reduce compulsory misses but raise the miss penalty.
- **Write policies:** write-through (writes go to memory too; simple, slower; often with a write buffer) and write-back (set the dirty bit, write the block back on eviction; less traffic). Write-allocate vs no-write-allocate.

Average access time

Single level: AMAT = hit time + miss rate × miss penalty
Two levels: AMAT = $t_1 + m_1 \times (t_2 + m_2 \times t_mem)$, where m_2 is the *local* miss rate of L2
Global miss rate of L2 = $m_1 \times m_2$ · CPI with memory stalls = base CPI + (memory accesses per instruction) × miss rate × miss penalty

- Example: L1 1 ns, 90% hits; L2 10 ns, 80% local hits; memory 100 ns → $1 + 0.1 \times (10 + 0.2 \times 100) = 4$ ns.
- Simultaneous (parallel) access to levels vs hierarchical (sequential) access changes the formula: read the question for which is used.
- For trace questions: compute index and tag for each address, track which block sits in each line, count hits and misses. Sequential array accesses with block size B hit B−1 of every B accesses (when the data fits).
- Main-memory organisation: DRAM (capacitor, needs refresh, cheaper) vs SRAM (flip-flops, faster, used for cache). Interleaved memory: n banks give up to n× bandwidth for sequential accesses. ROM, PROM, EPROM, EEPROM, flash.
- Virtual memory caches: TLB (see OS notes), virtually-indexed physically-tagged cache.
- Memory chip sizing: a memory of 2^a words × w bits needs a address lines and w data lines; building a large memory from small chips: (total size / chip size) chips, with decoders on the high-order address bits.

2. Pipelining, Hazards, CPI

- Classic 5-stage RISC pipeline: IF, ID, EX, MEM, WB. Ideal: one instruction completes per cycle after the pipeline fills; CPI = 1.
- **Time for n instructions on a k-stage pipeline** = (k + n − 1) cycles. Non-pipelined: n·k cycles (equal stages). **Speed-up** = n·k/(k + n − 1) → k for large n. The clock period is set by the slowest stage plus latch delay: T_clk = max(stage delays) + latch overhead.
- Example: 5 stages, 2 ns each, 100 instructions → (5 + 99) × 2 = 208 ns.
- With stalls: CPI = 1 + average stall cycles per instruction. Speed-up = (CPI_unpipelined × T_unpipelined)/(CPI_pipelined × T_pipelined) = (5 × 2.5 ns)/(1.25 × 0.78 ns).
- **Hazards:**
  • **Structural**: two instructions need the same hardware (e.g. one memory port). Fix: duplicate resources.
  • **Data:** RAW (true dependence, the common one), WAR and WAW (name dependences, appear with out-of-order execution). Fix: **forwarding (bypassing)**; a load followed immediately by a dependent instruction still needs one stall (load-use hazard). Without forwarding, a dependent instruction waits until the write-back completes (typically 2 stall cycles if a register can be written and read in the same cycle).
  • **Control:** branches; fix by stalling, predicting not-taken, branch prediction, delayed branches. Branch penalty = number of stages before the branch resolves; CPI = 1 + branch frequency × penalty × misprediction rate.
- Count stalls by drawing the pipeline diagram instruction by instruction.
- **Superscalar** (several instructions issued per cycle), **VLIW**, out-of-order execution, register renaming, dynamic scheduling (Tomasulo), speculative execution.
- **Performance:** CPU time = instruction count × CPI × clock period. MIPS = clock rate / (CPI × 10^6). Amdahl's law: speed-up = 1/((1 − f) + f/s) where f is the enhanced fraction and s the speed-up of that fraction. Throughput vs latency: pipelining improves throughput, not the latency of one instruction.

> **Tip:** In pipeline stall questions, note whether the register file allows write-then-read in the same cycle (half-cycle trick) and whether forwarding exists. These change the stall counts.

3. Instruction Set, Addressing Modes, Data Path, Control

- **Instruction format:** opcode, address fields. With a total instruction length L: immediate field width = L − opcode − register fields. Number of distinct opcodes = 2^(opcode bits); registers addressed with $\log_2$(registers) bits.
- **Expanding opcodes:** use unused bit patterns of the opcode field to encode instructions with fewer operands; count instructions by levels.
- **Instruction types by operands:** 3-address (r = a op b), 2-address, 1-address (accumulator), 0-address (stack machine, operations on top two stack items; postfix evaluation). Fewer addresses → more instructions to evaluate an expression.
- **Addressing modes:** immediate (operand in instruction), direct (address in instruction), indirect (address of address), register, register indirect, indexed (EA = base + index register), base-register (relocation), relative (PC + offset, for branches and position-independent code), auto-increment/decrement, stack. Effective address (EA) questions: apply the mode to the given register contents. Number of memory accesses: immediate 0, register 0, direct 1, indirect 2, register indirect 1.
- **PC-relative branch:** target = (address of the next instruction) + offset × (instruction size, if the offset counts words). Range = ±2^(bits − 1) instructions. Example: 12-bit signed offset in words, 4-byte instructions: max forward target = PC + 4 + 2047 × 4.
- CISC (complex, variable-length, many addressing modes, microcoded) vs RISC (simple, fixed-length, load/store architecture, hardwired control, many registers, easy to pipeline).
- **Data path:** register file, ALU, memory, PC; buses; multiplexers and control signals (RegWrite, ALUSrc, MemRead, MemWrite, MemtoReg, Branch). For R-type, load, store and branch instructions, know which control signals are set. Single-cycle design: clock period = the longest instruction path (usually load).
- **Control unit:** hardwired (fast, difficult to change) vs microprogrammed (control store of microinstructions; flexible, slower). Horizontal microcode: wide words with one bit per control signal (more parallelism, larger control store); vertical: encoded fields (smaller, slower to decode). Number of control-store bits = number of microinstructions × width.
- **Subroutine calls** use a stack for the return address; interrupts save the PC and flags. Instruction cycle: fetch, decode, execute, (memory), write-back.
- Integer and floating-point arithmetic details: see the Digital Logic notes.

4. I/O: DMA and Interrupts

- I/O techniques: **programmed (polling)** — CPU busy-waits; **interrupt-driven** — device interrupts when ready, the CPU does other work in between, with ISR overhead for every transfer; **DMA** — a DMA controller moves blocks between device and memory without the CPU; one interrupt at the end.
- **Memory-mapped I/O:** device registers share the memory address space (no special I/O instructions, uses part of the address space); **isolated (port) I/O:** separate address space and IN/OUT instructions.
- **DMA modes: burst (block)** — the controller holds the bus for the entire block; **cycle stealing** — takes the bus for one cycle at a time between CPU cycles; **transparent** — uses the bus only when the CPU does not need it. DMA steps: CPU sets the address, count, direction; device requests; controller gets the bus (HOLD/HLDA), transfers; interrupt on completion.
- **CPU time stolen by cycle stealing** = (number of transfers per second) × (bus cycle time). Example: 2 MB/s with 4 bytes per transfer = 5×10^5 transfers/s; at 100 ns each → 5% of the time.
- **Polling overhead** = polls per second × time per poll. **Interrupt overhead** = event rate × interrupt-service cost. Interrupts win at low event rates; polling wins at very high rates.
- **Interrupts:** maskable vs non-maskable; vectored (the device supplies the ISR address) vs non-vectored; priority resolution with daisy-chaining or a priority encoder. Sequence: finish the current instruction, save PC and flags, disable interrupts, jump to the ISR, restore on return.
- Bus: synchronous (clock-driven) vs asynchronous (handshake). Data transfer rate of a bus = width × frequency. Serial vs parallel; USB, SATA; bus arbitration (daisy chain, polling, independent requests).

5. Exam strategy & revision checklist

- Study order: cache (address bits, AMAT) → pipeline (cycles, stalls, speed-up) → instruction formats and addressing → DMA/interrupts.
- Most questions are NAT: write each formula, plug in numbers, and check units (ns vs µs, bytes vs bits).
- ☐ Tag/index/offset bits. ☐ Two-level AMAT. ☐ $k + n - 1$ pipeline cycles and hazard stall counting. ☐ Immediate field width and opcode expansion. ☐ EA for each addressing mode. ☐ DMA cycle-stealing percentage. ☐ Redo `GATE_CS_Computer_Organization.pdf` past questions.